# Data Challenge 2

## Mathematical Models

*Author*

Paul Beggs
BeggsPA@Hendrix.edu

*Instructor*

Dr. Christopher Camfield

*Due*

March 6, 2026

# 1 Introduction

When trying to find the temperature of a room, we often take many samples and average them together to achieve a global average. However, to ensure average is appropriately sampling the whole room, we take many recordings. For this problem, we take 500 temperature samples of a $20 \times 10 \times 8$ room, showcased in Figure 1. In an ideal situation, we would have evenly sampled temperatures across the whole room, so we know that when we average them, we are appropriately sectioning off the room into equally representable sections. However, in this paper, we walk through an approach of finding the average temperature when the sampling is *uneven*.

# 2 The Algorithm

## 2.1 Overview

As mentioned in the introduction, our room consists of 500 unevenly sampled temperatures, each with an associated $x$, $y$, and $z$ coordinate. Because our room is unevenly sampled, we fill the room with 216 rectangular prisms (or simply a "box"). Then, we account for every box's boundary, and add the contained temperatures to a list associated with that box. Once we have every temperature in the box accounted for, we find the average temperature, and then multiply it by the reciprocal of the total boxes. This achieves a weighting effect that reduces the impact of boxes that are overcrowded, and bolsters those that are sparse. If a box is empty, then we assume the temperature to be equal to the average of temperature values, 54.96. We do this for every box, and add each box's weighted temperature together to achieve a weighted global average temperature of 55.24. The rest of this section goes into the details of this summary.

## 2.2 Initialization & Sorting

First, our goal is to set up the boxes so that we have an equal partition that fills the room. We do this by partitioning each dimension into intervals, denoted in the initialization algorithm as $n_{bins}$. For example, if we set $n_{bins} = 6$, then we would have box lengths along the $x$-axis of $bl_x = 10/3$ because the total length is of the room is 20. So, we define how many intervals we want, so we can create boxes that have equally distributed lengths along the whole of the room. Finding the optimal number for $n_{bins}$ came down to balancing the number of empty boxes to ones that had values. We settled on 6 because it left us with only 21 boxes that were empty, thus allowing for an appropriate amount of partitioning, while not letting the global average dominate the weighted average.

Since we have 3 dimensions and 6 bins, we end up with 216 total boxes that we can assign temperatures to. To assign temperatures to each box, we have to know which box that temperature is in. So, we start by calculating the box lengths for every dimension in the room. Then, for each coordinate in the data set, we divide it by the respective box length and round down. This forces the coordinate into one of the 0 to $n_{bins} - 1$ bins. To account for points that may fall on the room's border, we force them into the last bin available.

To put the assigning temperatures algorithm into context, we walk through 2 examples with $n_{bins} = 6$, so $bl_x = 10/3$, $bl_y = 5/3$, and $bl_z = 4/3$. The first example has coordinates $(5.3, 4.2, 3.0)$ with associated temperature 43.702245. We divide each by the box lengths to get $i = 1$, $j = 2$, and $k = 2$. Thus, we know this coordinate falls within the bounds of this indexed box, so we add the temperature. The second example is to do with a point that is on the border. Note that, for our room with dimensions $20 \times 10 \times 8$, we actually don't have any points that fall on the border, but it is good practice to have the check in place. So, let point $(20.0, 9.5, 8)$ be in the room. We would calculate $i = \lfloor 20/3.33 \rfloor = 6$, $j = \lfloor 9.5/1.66 \rfloor = 5$, and $k = \lfloor 8/1.33 \rfloor = 6$. Notice that indices $i$ and $k$ are not included in the range $\{0, 1, \ldots, n_{bins} - 1\}$. Thus, this is why we included the min(...) check in the formula. At the end of the loop, we assign the temperature to the associated box.

### 2.3 Calculating the Weighted Average

Now that we have every temperature assigned to a box, we can calculate the global weighted temperature for the room, as shown in the calculation algorithm. Now, each box either has one or more temperatures, or none at all. So, if the box has more than one temperature, we calculate the average in the box; however, if the box is empty, then we calculate the average of *all* temperatures in the room, and assign the box that value. At the end, we do a weighted sum of every box's recorded temperature by multiplying the average by a scaling value, the reciprocal of the number of boxes. This allows for equality among each box, so not one box dominates the weighted average. For this problem, we find a total weighted average of 55.24 degrees, or just 0.28 degrees higher than the global average.

## 3 Conclusion

In closing, we note that while our approach limits the problem of uneven sampling, there remains room for improvement. One of the main problems of our model is the assumption that empty boxes take on the global average of the room. It is more likely that an empty space shares the temperature of its immediate surroundings rather than the average of the entire room. Additionally, there are much more sophisticated approaches such as an Octree algorithm or Voronoi tessellation, which solve the empty box problem entirely.

Despite this limitation, the binning approach provides a more accurate representation of the room's temperature than a naive global average. Because our 500 sensors are unevenly distributed, the arithmetic average would be disproportionately skewed by regions with a high density of sensors. By partitioning the space into 216 equal-volume boxes and averaging the temperatures within them first, our algorithm guarantees that every region of the room has an equal weight in the final calculation.

**Algorithm 1** Partition initialization

```
procedure MakeBoxes(n_bins)                    ▷ Initialize an empty box for all points
    for i ∈ {0, 1, ..., n_bins − 1} do                                  ▷ n_bins ∈ ℤ⁺
        for j ∈ {0, 1, ..., n_bins − 1} do
            for k ∈ {0, 1, ..., n_bins − 1} do
                boxes[(i, j, k)]
            end for
        end for
    end for
end procedure
```

**Algorithm 2** Assigning temperatures

```
procedure FillBoxes(xs, ys, zs, temps, n_bins)
    bl_x ← rl_x / n_bins                  ▷ bl_x is the box length along the x-dimension
    bl_y ← rl_y / n_bins                  ▷ rl_y is the room length along the y-dimension
    bl_z ← rl_z / n_bins
    for x, ∈ xs, y ∈ ys, z ∈ zs, t ∈ temps do          ▷ Find temperature's coordinates
        i ← min(⌊x/bl_x⌋, n_bins − 1)
        j ← min(⌊y/bl_y⌋, n_bins − 1)
        k ← min(⌊z/bl_z⌋, n_bins − 1)
        boxes[(i, j, k)] ← t                                ▷ Add temperature to its box
    end for
end procedure
```

**Algorithm 3** Calculate global temperature through weighting

```
procedure WeightTemperature(boxes)                  ▷ Get the room's temperature
    for b_temps ∈ boxes do                          ▷ For each box's list of temperatures
        if length(b_temps) > 0 then
            box_avg ← sum(b_temps)/length(b_temps)                ▷ Average this box's temps
        else
            box_avg ← sum(all_temps)/length(all_temps)   ▷ Global average temp as substitute
        end if
        total_temp ← box_avg · (1/n_boxes)          ▷ Add weighted temperature to total temp
    end for
end procedure
```

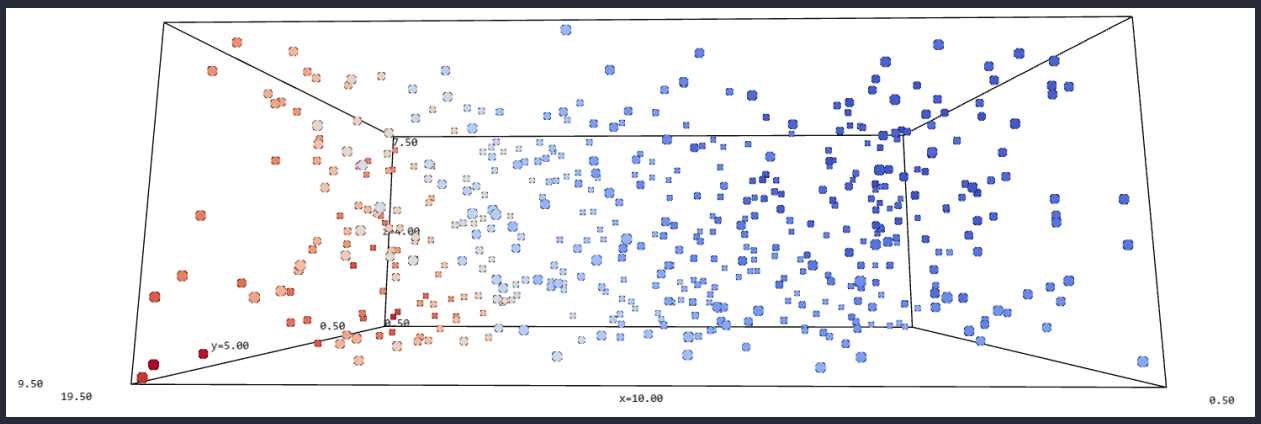


Figure 1: Implicit plot of all 500 color coded points. Red indicates high values, blue indicates low values.